Module 4 (Regression Testing and Test Management)

Importance of regression testing in software maintenance, Regression test planning and case selection,
Dynamic slicing and test minimization techniques, Tools for regression testing, Test planning, execution,
and reporting.

- Regression testing is the process of re-executing test cases after **code changes** (bug fixes, enhancements, patches) to ensure that **existing functionality is not broken**.
- Software maintenance accounts for a major portion of the software lifecycle. During maintenance, changes are inevitable, and regression testing plays a critical role in maintaining system integrity.
- Regression testing ensures:
    - Stability of the existing system
    - Reliability of the software after modifications
    - Continuous quality across versions

Why Regression Testing Is Important

1. Prevents Introduction of New Defects

Code changes in one module may unintentionally affect other dependent modules. Regression testing helps detect these **side effects** early.

2. Ensures Functional Consistency

Regression testing confirms that **existing features continue to behave as expected**, even after enhancements or bug fixes.

3. Supports Frequent Releases

In agile and DevOps environments, software is released frequently. Regression testing ensures that **rapid changes do not compromise quality**.

4. Reduces Business Risk

Failures in production can lead to financial loss and customer dissatisfaction. Regression testing reduces the risk of **post-release failures**.

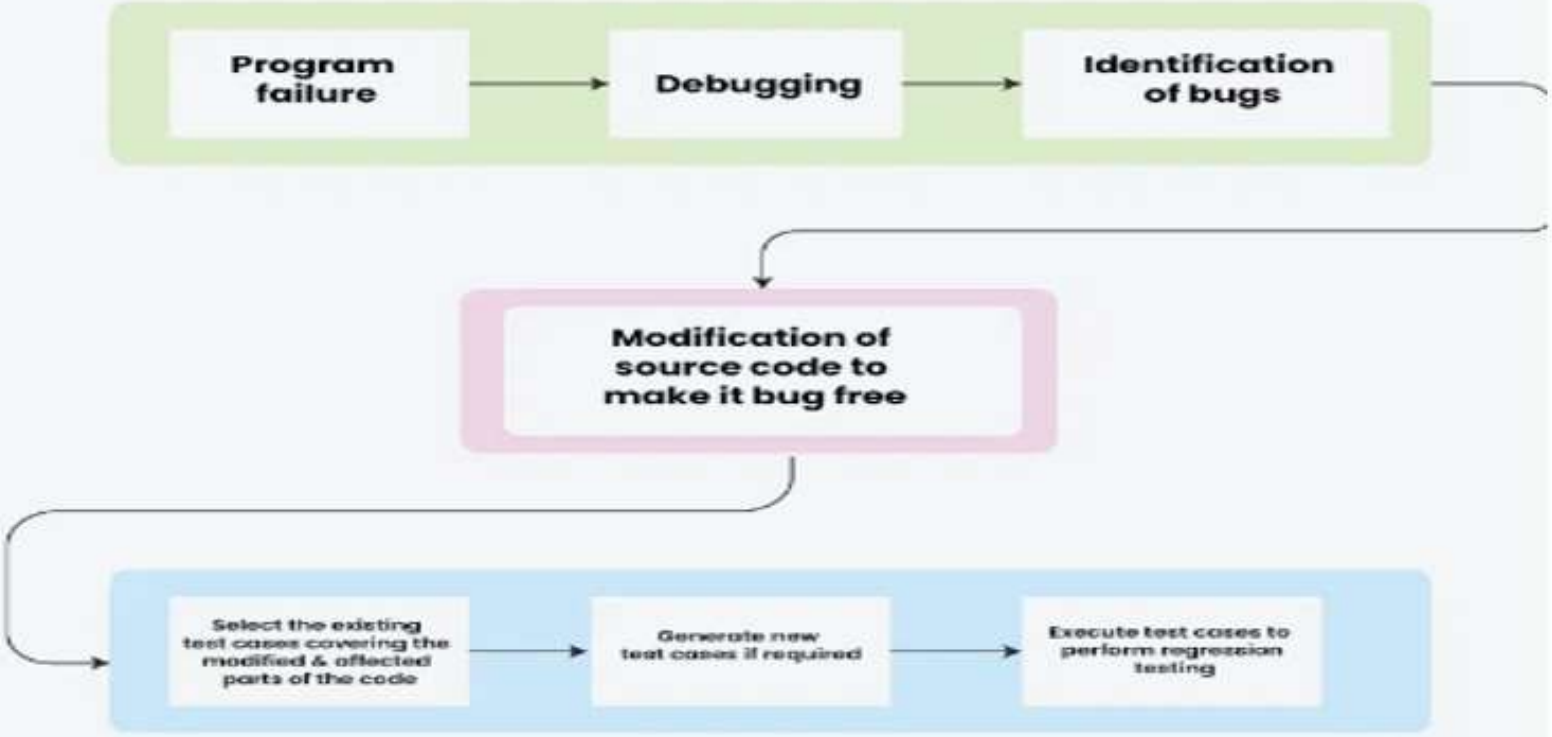
5. Improves Software Reliability

By continuously validating core functionalities, regression testing increases **confidence in software stability**.

selective re-testing

- Regression testing follows **selective re-testing technique**.
- When defects are fixed, the test team selects only the relevant test cases to verify those fixes. An **impact analysis** is performed to identify other areas that might be affected by the changes. Based on this analysis, additional test cases are selected to test the impacted areas. Since most of these test cases were already executed earlier, this method is called selective re-testing. Sometimes, new test cases may be created if needed, but in most cases, regression testing mainly **reuses existing test cases** to ensure previously tested features still work correctly.

Process Regression testing



TYPES OF REGRESSION TESTING

There are two types of regression testing in practice.

1. **Regular regression testing**: A regular regression testing is done between test cycles to ensure that the defect fixes that are done and the functionality that were working with the earlier test cycles continue to work. A regular regression testing can use more than one product build for the test cases to be executed.

2. Final regression testing: **Final regression testing** is performed to verify the final product before release. The configuration management (CM) engineer provides the final build exactly as it will be delivered to the customer. This testing is carried out for a fixed time period agreed upon by both development and testing teams, called **cook time**.

During cook time, the product is continuously tested to identify defects that appear only after long usage, such as memory leaks. Some test cases are repeated to ensure there are no failures in the final build. All defect fixes must be completed before this testing begins.

Final regression testing is very critical because it ensures that the **same tested build** is released to the customer.

WHEN TO DO REGRESSION TESTING?

- When **changes or enhancements** are made to the software
- After **defect fixes** are implemented
- Between **test cycles or builds**
- When fixes may cause **side-effects** in existing functionality
- After a **significant amount of testing** is already completed
- When a **large number of defects** have been resolved
- Before **closing defects**, to ensure no new issues are introduced
- Periodically, as a **preventive (proactive) measure**
- Before **final release**, during final regression testing

HOW TO DO REGRESSION TESTING?

1. Performing an initial “Smoke” or “Sanity” test
2. Understanding the criteria for selecting the test cases
3. Classifying the test cases into different priorities
4. A methodology for selecting test cases
5. Resetting the test cases for test execution
6. Concluding the results of a regression cycle

1. Performing an Initial “Smoke” or “Sanity” Test

- Whenever changes are made to a product, it is important to first ensure that **basic functionality works**.
- For example, a database build should:
  - Start successfully
  - Perform basic operations (queries, data definition, data manipulation)
  - Shut down properly
- Key interfaces with other products should also work correctly.
- These basic checks must be completed **before detailed testing** begins.
- If a build fails basic operations, further testing is meaningless until the issue is fixed.
- Developers are often required to run smoke tests successfully **before committing code** to the repository.
- Defects can be introduced not only by code changes but also by **build and configuration scripts**.
- Smoke testing helps detect such build-related issues early.
- Studies show that **around 15% of defects** are caused by build or configuration management processes.

Smoke Testing Consists of:

- Identifying **basic functionalities** that the product must support
- Designing test cases for these basic features
- Grouping them into a **smoke test suite**
- Running the smoke test suite on **every new build**
- Escalating to developers if smoke tests fail, and fixing or rolling back changes

2. Understanding the Criteria for Selecting the Test Cases

Understanding Test Case Selection for Regression Testing

- After **smoke testing**, the product is ready for detailed testing.
- Regression testing aims to:
    - Verify that **defect fixes work**
    - Ensure **no unintended side-effects** are introduced

Approaches to Select Regression Test Cases

1. Fixed (Constant) Test Set

- Same set of test cases run for every build
- Simple to execute
- Limitations:
    - Unnecessary tests may be run
    - New defects may not be covered
    - Not optimal for all changes

2. Dynamic Test Case Selection (Preferred)

- Test cases selected based on current changes
- Requires understanding of:
    - Defect fixes in the current build
    - How to test those changes
    - Impact on other system areas
    - How to test impacted areas

Criteria for Selecting Regression Test Cases

- Test cases that found **maximum defects in the past**
- Test cases related to **changed functionality**
- Test cases where **issues were previously reported**
- Test cases covering **core / basic functionality**
- **End-to-end test cases**
- **Positive test cases**
- Test cases for **highly visible user features**

Best Practices in Test Case Selection

- Avoid selecting too many **irrelevant or failing test cases**
- Prefer **positive test cases** over negative ones in final regression
- Negative test cases may confuse failure analysis
- Regular test cycles should include both positive and negative tests

3. Classifying Test Cases

- the test cases can be classified into various priorities based on importance and customer usage.

  Priority-0 These test cases can be called **sanity test cases** which check basic functionality and are run for accepting the build for further testing. They are also run when a product goes through a **major change.** These test cases deliver a very high **project value** to both to product development teams and to the customers.

  Priority-1 Uses the **basic and normal setup** and these test cases deliver high project value to both development team and to customers.

  Priority-2 These test cases deliver **moderate project value.** They are executed as **part of the testing cycle** and selected for regression testing on a **need basis.**

4. Methodology for Selecting Test Cases

- After test cases are classified by **priority**, they can be selected for regression testing.
- Regression testing approaches vary and are chosen on a **case-by-case basis**.
- The discussed method considers:
    - **Criticality of defect fixes**
    - **Impact of defect fixes**
- There are several methodologies available in the industry for selecting regression test cases.

Case 1: Low Criticality & Low Impact

- Execute **few test cases** from the Test Case Database (TCDB)
- Test cases can be from **Priority 0, 1, or 2**
- Minimal regression effort is sufficient

Case 2: Medium Criticality & Medium Impact

- Execute **all Priority-0 and Priority-1** test cases
- Select **few Priority-2** test cases if needed
- Priority-2 tests are **optional but desirable**

Case 3: High Criticality & High Impact

- Execute **all Priority-0 and Priority-1** test cases
- Execute a **carefully selected subset of Priority-2** test cases
- Ensures maximum coverage and reduced risk

Challenges of Impact-Based Selection

- Requires **impact analysis** for every defect fix
- Can be **time-consuming**
- Alternative methods may be used when:
    - Time is limited
    - Risk is low

Alternative Regression Testing Methodologies

- **Regress All**
  - Execute all Priority-0, 1, and 2 test cases
- **Priority-Based Regression**
  - Execute test cases in priority order until time runs out
- **Regress Changes**
  - Select test cases based on **code changes**
- **Random Regression**
  - Randomly select and execute test cases
- **Context-Based Dynamic Regression**
  - Start with Priority-0 tests
  - Select additional tests based on execution results

5 Resetting the Test Cases for Regression Testing

- After selecting the regression test cases, the next step is to **prepare them for execution**.
- For this, a **test case result history** is required.
- In large products, testing happens in many cycles. It is important to track:
  - Which test cases were executed
  - In which build or test cycle
  - What the results were

This information is stored in the **Test Case Database (TCDB)** and is called **test case result history**.

Test Case Result History

- Test case result history records:
  - Test case execution details
  - Pass/fail status
  - Build number and cycle
- It helps testers know:
  - What was tested earlier
  - What still needs to be tested
- Not all test cases are executed in every test cycle
- Hence, history becomes very useful for regression testing

What is Resetting a Test Case?

- **Resetting a test case** means marking it as:
    - **“Not Run”** or **“Execute Again”**
- This is done by setting a **reset flag** in TCDB
- When a test case is reset:
    - Previous results are hidden
    - Tester executes it like a new test case
- This avoids bias caused by knowing past results

Why Reset Test Cases?

- Defect fixes may introduce **side-effects**
- Resetting ensures testers:
  - Re-evaluate the test case carefully
  - Select test cases based on **impact of changes**
- Risk is higher when:
  - Fixes are done close to release
- In such cases, **more test cases must be reset and executed**

When Should Test Cases Be Reset?

Test cases should be reset in the following situations:

1. Major changes in product functionality
2. Changes in build or configuration procedures
3. Long release cycles where tests were not executed recently
4. Final regression testing with selected test cases
5. When expected results may differ from previous cycles
6. Test cases related to defect fixes or production issues
7. When existing functionality is removed
8. Test cases that always pass can be reset or removed
9. Negative test cases that never detect defects can be removed

Reset vs Rerun Test Cases

Reset

- Indicates **medium to high risk**
- Previous results are hidden
- Results may change
- Important for release decision
- Executed first before release

Rerun

- Indicates **low risk**
- Results expected to remain same
- Used to gain confidence
- Not expected to reveal major issues

Reset Based on Priority Stability

- If **Priority-0** test cases have >95% pass rate:
    - Do not reset unless there is a major change
- Similarly:
    - Do not reset Priority-1 unless moving to Priority-2 phase
- Reset depends on **stability of functionality**

Reset in Component Test Phase

- Regression testing uses only **Priority-0** test cases
- For every new build:
    - All Priority-0 test cases are **reset**
- Testing starts only when:
    - All Priority-0 test cases pass

Reset in Integration Test Phase

- Execute **Priority-0 and Priority-1** test cases
- Reset is done only when:
    - Impact and criticality of fixes are high
- Reset may affect:
    - Priority-0 and Priority-1 test cases

Reset in System Test Phase

- Priority-2 testing starts after acceptable Priority-1 pass rate
- Reset is done only when:
    - Impact of changes is **very high**
- Reset may affect:
    - Priority-0, Priority-1, and Priority-2 test cases

Why Resetting is Important in Regression Testing

- Shows **actual remaining testing effort**
- Prevents false reporting of test completion
- Removes assumption:

  “If it passed earlier, it will pass again”

- Regression testing assumes:

  Future is not an extension of the past

- Resetting ensures unbiased and effective regression testing

Concluding the Results of Regression Testing

Regression testing is usually performed **after test cycles** and often **close to the release date**.
Because of this, regression test results are monitored by:

- Test managers
- Developers
- Project managers
- Release and quality teams

Regression results help everyone understand:

- Whether **defects still exist**
- Whether **defect fixes are working correctly**

Why Regression Results Are Important

- Regression testing reuses test cases that have already been executed earlier
- These test cases are expected to **pass again**
- Ideally, the **pass percentage should be 100%**
- Any failure indicates:
    - A side-effect
    - An incomplete or incorrect defect fix

Expected Outcome of Regression Testing

- If defect fixes are correct:
    - All regression test cases should pass on the same build
- If pass percentage is **less than 100%**:
    - Test manager compares results with **previous builds**
    - Based on comparison, a conclusion is made

Regression Test Planning and Case Selection

- When performing regression testing in the software maintenance phase, proper planning and case selection are essential to ensure that the testing process is efficient, thorough, and focused on the right areas.

1. Regression Test Planning

- Regression test planning is a structured process that **outlines the scope, objectives, and logistics of regression testing.** Here are the key steps involved in creating a solid plan:

a. Define the Scope of Regression Testing

- Identify which changes require testing: Determine whether the new updates or bug fixes could affect critical areas of the application. This includes new features, defect fixes, code refactoring, or any changes to the underlying infrastructure.
- Prioritize areas to be tested: Focus on areas with higher complexity or more frequent changes that have a higher chance of introducing defects.

b. Determine Testing Environment and Tools

- Environment setup: Ensure that testing is done on environments that mirror production as closely as possible.
- Tools for automation: If using automation, select appropriate regression testing tools (like Selenium, JUnit, TestComplete, etc.) to speed up execution and reduce human error.

c. Define Test Data and Test Execution Strategy

- Prepare test data: Ensure that data used in testing scenarios is representative of real-world situations and covers all edge cases.
- Testing approach: Decide whether the regression testing will be manual or automated, or a combination of both. Automated tests can run frequently and save time, while manual tests may be necessary for complex user interactions or visual checks.

Regression Test Case Selection

- Selecting the right test cases is critical to ensure that regression testing is both thorough and efficient.
- The goal is to identify the areas most likely to be impacted by the changes and avoid unnecessary testing.

Types of Regression Test Cases to Include

- Test cases that cover core functionalities: These are the most critical features of the application that the users rely on (e.g., login, payments, data processing, etc.).
- Test cases related to recent changes: Focus on the parts of the software that were modified. For example, if the code for the payment module was updated, prioritize tests that validate payments.

- Test cases for high-risk areas: These include areas that have experienced issues in the past, are complex, or involve integration with external systems.
- Boundary and edge case tests: These cases often reveal bugs that are hard to catch with typical tests, such as handling of extreme values or unexpected inputs.
- Automated test cases: For faster execution, it's best to have automated tests for frequently used functionality.

Test Case Selection Techniques

- Risk-based selection: Focus on areas with the highest likelihood of failure and the greatest potential impact on the user experience.
- Change-based selection: Select test cases that test areas of the system that were directly impacted by the recent code changes or fixes.
- Code coverage-based selection: Ensure that the regression suite provides sufficient code coverage, especially if test automation is used. Tools like JaCoCo (for Java) can help identify areas of the codebase that haven’t been tested.
- History-based selection: If available, look at historical data of past defects to focus on test cases related to areas where defects were frequently found.

Prioritize Regression Test Cases

- Critical business functions: Test the functionalities that directly impact business operations or users.
- Frequently used features: Focus on areas that are frequently used or interact with other parts of the system.
- Areas with complex or recent changes: Modules or features that were modified recently or are difficult to test automatically should be given higher priority.

Eliminate Redundant or Outdated Test Cases

- Remove obsolete tests: Over time, some tests may no longer be relevant.
- As the software evolves, some older tests may no longer serve their intended purpose and should be removed or refactored.
- Automate repetitive test cases: Repetitive tests, especially for minor changes, should be automated so that testers can focus on more exploratory or complex scenarios.

Test Case Documentation

- Document the selected test cases properly, including:
- Test Case ID: Unique identifier for each test.
- Test Description: What the test is validating.
- Preconditions: Any setup required before the test runs.
- Steps to Execute: Step-by-step instructions for executing the test.
- Expected Result: What should happen if the system behaves correctly.
- Postconditions: The state of the system after the test.

Dynamic Slicing and Test Minimization Techniques

- In the context of software testing, **both dynamic slicing and test minimization are techniques used to optimize the testing process**, ensuring that tests are efficient and cover the necessary portions of the software while avoiding redundancy.
- These techniques are especially important in large or complex systems, where comprehensive testing can become costly and time-consuming.

Dynamic Slicing

- Dynamic slicing is a program analysis technique used to identify and isolate portions of a program (called "slices") that are relevant to a particular computation or output.
- It differs from static slicing because it takes the actual execution (runtime) data of the program into account.

How Dynamic Slicing Works

- In dynamic slicing, the "slice" of the program is based on the actual inputs and execution paths during a particular run of the program.
- The slice includes the subset of statements that directly or indirectly contribute to the output for a given input.
- Unlike static slicing, which is determined by the program's structure and logic, dynamic slicing considers the specific data flow that happens when the program is executed.

Steps in Dynamic Slicing

- Execution Tracing: During program execution, a trace of variables, statements, and their values is generated.
- Identify Relevant Portions: Once a specific program output or bug is identified, dynamic slicing helps isolate the part of the program that influences that particular output.
- Slice Generation: The slice includes all the statements that influence or depend on the variables contributing to the output.
- Analysis: Once the slice is identified, you can analyze or test only those parts of the program, which helps in debugging or regression testing.

Advantages of Dynamic Slicing

- Efficient Debugging: Helps isolate the root cause of defects by identifying the minimal subset of code responsible for a specific behavior.
- Reduced Test Case Size: By focusing on the relevant portions of code for a given input, dynamic slicing reduces the number of test cases needed to check specific functionality.
- Improved Testing Focus: It allows testers to focus on the parts of the system that actually contribute to the output for a specific test, avoiding unnecessary tests on unrelated code.

Limitations of Dynamic Slicing

- Execution Dependent: It requires actual program execution with specific inputs, so it's not as useful for analyzing all potential behaviors of the program without running it.
- Performance Overhead: Generating execution traces and analyzing them for dynamic slices can add overhead to the testing process.

Test Minimization

- Test minimization refers to the process of reducing the number of test cases in a test suite while maintaining the effectiveness and coverage of the tests.
- It aims to retain a set of tests that can validate the software's functionality while removing redundant or unnecessary tests.

How Test Minimization Works

- Test minimization techniques generally rely on the principle of redundancy elimination, ensuring that each test case adds unique value in terms of coverage or detection of faults.
- The goal is to create a minimal subset of test cases that can still provide sufficient coverage of the system.

Key Approaches in Test Minimization

- Redundancy Detection: Identify and remove test cases that do not add value, such as those that test the same functionality under the same conditions.
- Exact Redundancy: Test cases that are completely redundant because they test the same functionality with the same inputs.
- Approximate Redundancy: Test cases that test similar functionality and might not reveal additional faults but do not necessarily repeat the same steps.
- Covering Arrays: Select test cases that cover all combinations of inputs, ensuring minimal redundancy while achieving sufficient coverage. This can be especially useful for combinatorial testing, where testing every combination of variables might lead to an explosion in the number of tests.
- Use pairwise testing or t-wise testing to cover all combinations of input parameters.

- Test Case Prioritization: Prioritize tests that are more likely to detect faults based on historical defect data, risk analysis, or code coverage. This approach doesn't minimize the number of tests directly, but it ensures that the most valuable tests are executed first, which can help reduce the overall test time.

- Impact Analysis: Analyze which parts of the software the tests cover and eliminate tests that do not provide new information. This is especially important in regression testing, where old test cases may no longer be relevant after new features or code changes.

- Cluster-Based Minimization: Group similar test cases into clusters based on their functionality and remove duplicates from within each group. This can involve automated techniques for clustering based on input data or coverage criteria.

Advantages of Test Minimization

- Reduced Testing Time: By reducing the number of tests, test execution time and resources required are decreased, which can be crucial for large systems.
- Cost Efficiency: Minimizing the test suite reduces testing costs while still maintaining essential coverage.
- Faster Feedback: A smaller set of tests can be run more quickly, providing faster feedback on code changes, which is especially useful in agile or continuous integration environments.

Limitations of Test Minimization

- Potential Loss of Coverage: If not done carefully, minimizing tests may reduce coverage, potentially missing critical issues or edge cases.
- Manual Effort: While many techniques for test minimization are automated, some approaches, such as test case clustering, may still require manual input to ensure that the minimized test suite retains sufficient coverage.
- Trade-off with Fault Detection: A smaller test suite might not detect faults as effectively, particularly if key scenarios or combinations of conditions are removed.

Combining Dynamic Slicing and Test Minimization

- Dynamic Slicing for Test Minimization: By using dynamic slicing, you can identify and isolate the relevant portions of the code that need to be tested for specific inputs. Then, test minimization techniques can remove redundant test cases while ensuring sufficient coverage.

- Efficient Regression Testing: When performing regression testing, dynamic slicing can help identify which tests are still relevant based on the modified code. Test minimization can then be applied to ensure that only the essential tests are run, speeding up the process without compromising quality.

- Cost-Effective and Efficient: Combining these techniques allows teams to focus on the most critical aspects of the system, improving testing efficiency and reducing the overall testing effort.

Tools for Regression Testing

- Regression testing tools are primarily used to automate the process of re-running test cases to ensure that new changes do not break existing functionality. Here are some popular tools:

1. Selenium

- A widely used open-source tool for automating web browsers.

Features:

- Supports various programming languages (Java, Python, C#, etc.).
- Can be used for automating web applications across different browsers.
- Integrates well with other tools like Jenkins for continuous testing.
- Best For: Web application regression testing and browser compatibility testing.

- Browser Compatibility: Supports multiple browsers, including Chrome, Firefox, Safari, and Edge, ensuring tests can be run across different environments.
- Programming Language Support: Allows writing tests in various programming languages such as Java, Python, C#, Ruby, and JavaScript, providing flexibility for testers.
- Cross-Platform: Capable of running on different operating systems, including Windows, macOS, and Linux, which enhances the tool’s portability.
- Web Application Testing: Primarily designed for automating web applications, making it ideal for regression testing of web-based systems.

2. QTP (QuickTest Professional) / UFT (Unified Functional Testing)

- A commercial tool for automated functional and regression testing of web and desktop applications.

Features:

- Supports both functional and regression testing.
- Uses a script-based approach for automating tests (VBScript).
- Integration with ALM (Application Lifecycle Management).
- Best For: Regression testing for large enterprise applications.

3. TestComplete

- Purpose: A commercial automated testing tool for functional and regression testing.

Features:

- Allows for both script and scriptless test creation.
- Supports desktop, web, and mobile applications.
- Provides powerful object recognition features and integration with CI/CD tools.
- Best For: Automating UI and functional regression tests across different platforms.

4.JUnit / TestNG

- Purpose: These are frameworks for writing and running tests in Java (JUnit) and other JVM languages (TestNG).

Features:

- Easy integration with other tools such as Jenkins, Maven, or Ant.
- Support for test automation, parallel test execution, and creating custom test suites.
- Allows for running regression tests as part of the build process.
- Best For: Unit and regression testing in Java-based projects.

5. Appium

- Purpose: An open-source mobile application testing tool.
- Features:
- Supports Android and iOS mobile applications.
- Can automate both native and hybrid mobile apps.
- Supports multiple programming languages (Java, Python, JavaScript, etc.).
- Best For: Mobile app regression testing.

6. Ranorex

- Purpose: A test automation tool for desktop, web, and mobile applications.

Features:

- Offers both script-based and scriptless test automation.
- Has strong UI object recognition capabilities.
- Integrates with CI tools like Jenkins for continuous testing.
- Best For: Regression testing across a variety of applications

Tools for Test Planning

- Test planning tools help define the scope, objectives, and strategies for testing. They allow teams to organize their tests, assign tasks, and track progress.

1. TestRail

- Purpose: A test management tool that helps plan, track, and manage software testing efforts.

Features:

- Test case creation, execution, and result tracking.
- Integration with tools like Jira, Selenium, and Jenkins.
- Supports test plans, test suites, and custom reporting.
- Best For: Organizing test cases, planning, and tracking progress in large testing teams.

2. Zephyr

- A popular test management solution integrated with Jira.

Features:

- Allows creation and management of test cases, execution plans, and reporting.
- Can track test execution and defect management within Jira.
- Supports integration with CI tools and test automation tools.
- Agile teams already using Jira for project management.

3. Xray (Jira plugin)

- Purpose: A test management tool that integrates directly with Jira.

Features:

- Full lifecycle test management, from planning to execution and reporting.
- Supports both manual and automated tests.
- Allows test cases to be linked with requirements, user stories, and bugs.
- Best For: Teams using Jira that need full test management integration.

4. TestLink

- Purpose: An open-source test management tool for planning and tracking tests.

Features:

- Test case management, version control, and test execution tracking.
- Supports integration with Jira, Bugzilla, and other issue trackers.
- Provides comprehensive test reporting and analysis.

TEST PLANNING

- A well-defined test plan is essential for successful testing. It acts as a roadmap for the entire testing project.

Preparing a Test Plan:

- A test plan should cover:
- **Scope:** What will and will not be tested.
- **Strategy:** How testing will be performed, including tasks and methods.
- **Resources:** Required hardware, software, and personnel.
- **Timeline:** Schedule for testing activities.
- **Risks:** Potential issues and mitigation plans.

Scope Management: Deciding Features to be Tested/Not Tested:

Scope management involves:

- Understanding the product release.
- Breaking down the release into features.
- Prioritizing features for testing.
- Deciding which features to test and which to exclude.
- Estimating resources needed.

Prioritization factors:

- New and critical features.
- Features with catastrophic failure potential.
- Complex features.
- Extensions of defect-prone features.
- It's crucial to identify features and combinations that *won't* be tested, with justifications and risk assessments.

Deciding Test Approach/Strategy:

- This involves determining the appropriate types of testing for each feature or combination, including:
- Functional testing methods.
- Configurations and scenarios.
- Integration testing.
- Localization validations.
- Non-functional tests.

Setting up Criteria for Testing:

- **Entry Criteria:** Requirements that must be met before testing can begin.
- **Exit Criteria:** Conditions that indicate testing is complete.
- **Suspension Criteria:** Reasons for temporarily halting testing.
- **Resumption Criteria:** Conditions for restarting suspended tests.

- **Identifying Responsibilities, Staffing, and Training Needs:**
- Define roles and responsibilities clearly, ensuring accountability and preventing overlap. Staffing should be based on effort estimation, and training should address skill gaps.

- **Identifying Resource Requirements:**
- Estimate hardware, software, tools, licenses, and environmental needs (office space, support functions). Accurate estimation is crucial to avoid delays and demotivation.
- **Identifying Test Deliverables:**
- Define the outputs of the testing process, including:
- Test plans (master and individual).
- Test case specifications.
- Test cases (including automation).
- Test logs.
- Test summary reports.
- Defect repository updates.

Testing Tasks: Size and Effort Estimation:

- **Size Estimation:** Quantifies the amount of testing. Factors include product size (LOC, function points, screens/reports), automation extent, and platforms/environments. Use a Work Breakdown Structure (WBS).
- **Effort Estimation:** Determines the work required. Factors include productivity data, reuse opportunities, and process robustness. Expressed in person-days/months/years

Activity Breakdown and Scheduling:

- Translates effort into a timeline. Steps include:
- Identifying dependencies (internal and external).
- Sequencing activities.
- Estimating time for each activity.
- Monitoring progress.
- Rebalancing schedules and resources.

Communications Management:

- Establish procedures for effective communication, which will be discussed further in Section 15.3.

Risk Management:

- A cyclical process involving:
- Risk identification (checklists, history, networking).
- Risk quantification (probability and impact).
- Risk mitigation planning (alternative strategies).
- Risk response (when risks occur).

- Common testing risks include:
- Unclear requirements.
- Schedule dependence.
- Insufficient testing time.
- "Show stopper" defects.
- Lack of skilled testers.
- Inability to acquire test automation tools.
- **Summary:**
- Effective test planning is paramount for successful software testing. It requires a thorough understanding of the product, careful consideration of various factors, and proactive risk management. A well-defined test plan ensures that testing is focused, efficient, and contributes to a high-quality product release.

TEST PROCESS

Putting Together and Baselining a Test Plan:

- The test plan consolidates all planning aspects into a single document. A template is typically used and customized for each project.
- The plan undergoes review and approval by designated authorities.
- It's then baselined in the configuration management repository, serving as the foundation for the testing project. Changes to the plan require updates and re-baselining. Templates are also periodically reviewed and updated.

Test Case Specification:

- Based on the test plan, test case specifications are designed. A test case formally consists of:

1. **Purpose:** What feature/part is being tested (using consistent naming conventions).
2. **Items Under Test:** Versions/releases of the items.
3. **Environment:** Hardware, software (OS, database), and product setup (installation, configuration, data).
4. **Input Data:** Specific, unambiguous values for each field (e.g., "enter 789" instead of "a three-digit positive integer"). Use files for automated testing.
5. **Execution Steps:** Detailed instructions (for manual testing) or scripts (for automated testing). Detail level should match tester skill.
6. **Expected Results:** What the user should see (GUI, reports) and changes to persistent storage.
7. **Result Comparison:** "Intelligent" comparison of actual vs. expected results, accounting for acceptable differences (e.g., date, ID).
8. **Relationships to Other Tests:** Dependencies or reuse possibilities.

- **Update of Traceability Matrix:**
- The requirements traceability matrix is updated with test case identifiers, ensuring two-way mapping between requirements and tests.
- **Identifying Possible Candidates for Automation:**
- Decide which tests to automate based on:

1. Repetitive nature of the test.
2. Automation effort.
3. Manual intervention required.
4. Cost of the automation tool.

- **Developing and Baselining Test Cases:**
- Test cases are developed from specifications. This involves scripting for automated tests and detailed instructions for manual tests.
- Test cases should include change history documentation (what, why, who, when, how, affected files).
- All test artifacts (scripts, inputs, outputs) are stored in the test case database and SCM, requiring review and approval before baselining.

Executing Test Cases and Keeping Traceability Matrix Current:

- Test cases are executed at appropriate times (e.g., smoke tests daily, system tests during system testing).
- The defect repository is updated with fixed defects and new defects. It's the primary communication vehicle between test and development teams.
- Tests may be suspended based on defined criteria and resumed when conditions are met. Entry and exit criteria apply.
- The traceability matrix is updated during test design and execution. The matrix itself is under configuration management.

Collecting and Analyzing Metrics:

- Test execution data is collected and transformed into meaningful metrics .

Preparing Test Summary Report:

- A report summarizing the test cycle is generated, providing insights into product fitness for release.

Recommending Product Release Criteria:

- The testing team informs management about:

1. Existing defects.
2. Severity and impact of defects.
3. Risks of releasing with these defects.

- Management then makes a business decision about release, balancing the risks of releasing with defects against the risks of delaying the release to fix more defects. "Over testing" to remove every last defect can be as risky as "under testing."

- This section focuses on test reporting, a crucial communication mechanism within the software development process.

TEST REPORTING

- Effective communication is essential during testing. Two main types of reports are used:
- Test Incident Reports
- Test Summary Reports (Test Completion Reports)

Test Incident Report:

- These reports are generated throughout the testing cycle whenever defects are found.
- They are essentially entries in the defect repository.
- Each defect has a unique ID, which is used to track the incident.
- High-impact incidents are highlighted in the test summary report.

Test Cycle Report:

- Generated at the end of each test cycle (a series of tests using a specific product build), these reports provide:

1. Summary of activities performed during the cycle.
2. Defects uncovered, categorized by severity and impact.
3. Progress on defect fixes from the previous cycle.
4. Outstanding defects.
5. Variations in effort or schedule compared to the plan.

Test Summary Report:

- This report summarizes the results of a test cycle and provides a release recommendation. Two types exist:

1. Phase-wise test summary: Generated at the end of each testing phase.
2. Final test summary (Release Test Report): Contains details from all testing phases and teams.

- A summary report should include:

1. Summary of activities performed during the cycle/phase.
2. Variance from planned activities:
    1. Tests planned but not run (with reasons).
    2. Modifications to tests (TCDB should be updated).
    3. Additional tests run.
    4. Differences in planned vs. actual effort and time.
    5. Other deviations from the plan.
3. Summary of results:
    1. Failed tests with root cause descriptions.
    2. Severity and impact of defects.
4. Comprehensive assessment and release recommendation:
    1. "Fit for release" assessment.
    2. Release recommendation.

Recommending Product Release:

- The test summary report informs the decision to release the product.
- Ideally, the goal is zero defects, but market pressures may necessitate releasing with some defects.
- Release with defects is acceptable if management believes customer dissatisfaction risk is low.
- This decision is made by senior management after consulting with customer support, development, and testing teams to assess the overall organizational impact. Releasing with low-priority/low-impact defects, or defects triggered by unrealistic conditions, may be acceptable.